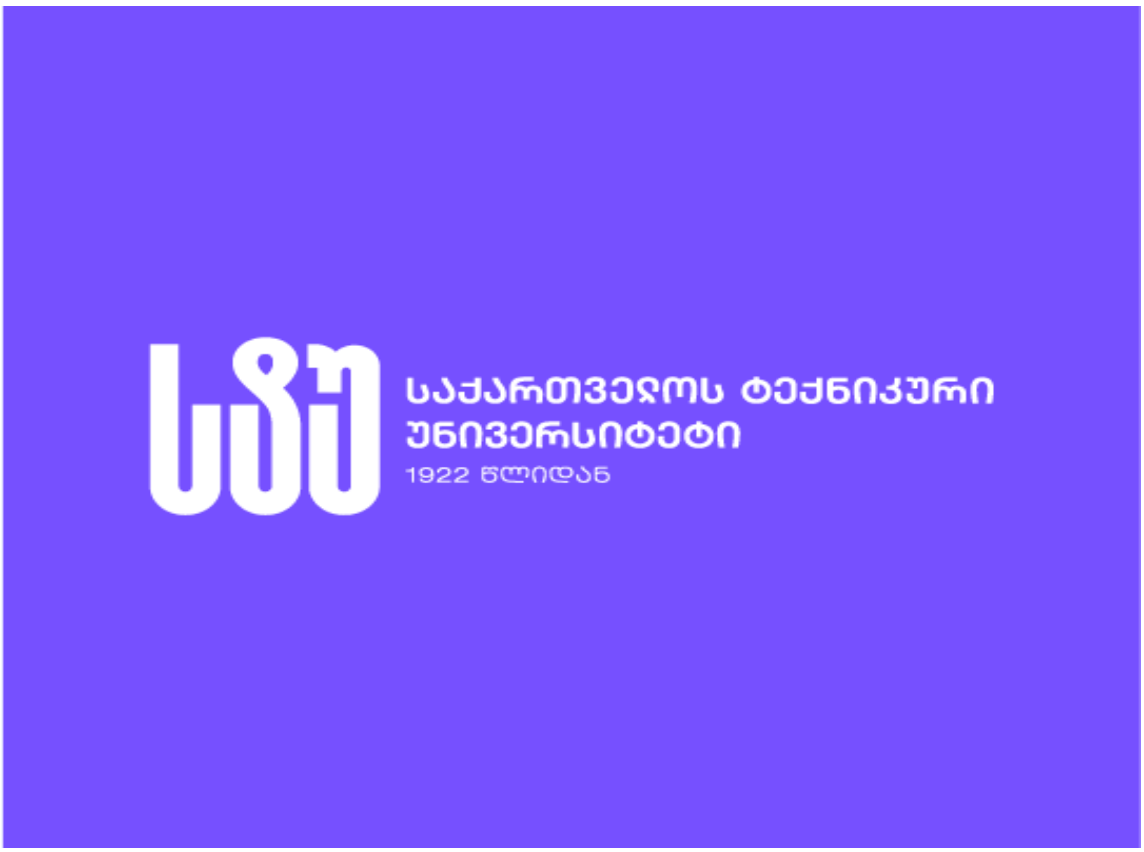

Name: Tamta Termakozashvili

Faculty: Informatics and Management Systems

Program: Computer Science (English)

Bachelor's Thesis

Design and Implementation of an Adaptive Memory System for LLM-Based Culturally-Aware AI Culinary Assistant with Multi-Language Support.

Georgian Technical University

Tbilisi, 0160, Georgia

February, 2026

Faculty of Informatics and Management Systems

Computer Science Program

Student Tamta Termakozashvili _____

name, surname Signature

Supervisor Mariam Chkhaidze _____

name, surname Signature

ABSTRACT

Large Language Models (LLMs) typically operate as static knowledge bases; once the training phase is complete, their internal parameters are frozen. Consequently, these models cannot naturally integrate new information or persist user-provided corrections across sessions. In the context of culturally specific domains—such as regional Georgian cuisine, where recipes vary significantly by family tradition and locality—this lack of runtime adaptation is a critical limitation. Without a persistence mechanism, a conversational agent cannot maintain the specific culinary constraints requested by a user, leading to factual inconsistencies and a diminished user experience.

This thesis presents the design and implementation of "Cheff Mshia," an AI-driven culinary assistant developed to overcome these limitations through an adaptive memory system. Rather than employing computationally expensive fine-tuning, the system implements a lightweight persistence layer using JSON-based storage. This architecture allows the application to capture user feedback in real-time and inject these corrections into the system prompt before each inference request, effectively simulating long-term memory and ensuring that user-specified preferences are prioritized over the model's baseline training data.

The development of the inference backend proceeded in two distinct phases. The initial prototype utilized FastAPI and TheMealDB to implement a strict ingredient-matching search. The system was subsequently migrated from the Gemini API to the Groq cloud infrastructure, utilizing the Llama-3.3-70b-versatile model. This migration resulted in a reduction of average response latency by over 70% and mitigated API rate-limiting errors observed under standard operational loads. To maintain linguistic stability, a custom language-detection module was implemented. This module utilizes Unicode character range verification to prevent unintended language switching during mixed-script inputs (Georgian and Latin), ensuring the response language remains consistent with the user's session settings.

The final application is deployed via Render and indexed through Google Search Console. Empirical load testing confirms that the file-based storage layer maintains a stable memory footprint of approximately 42MB and successfully ensures multi-tenant data isolation between concurrent user sessions.

Contents

Chapter 1

Introduction

1.1 Background and Motivation

The project originated as a web-based application designed to perform ingredient-based recipe retrieval, allowing users to identify viable dishes based on available household ingredients. While the initial prototype successfully implemented the ingredient-matching logic, subsequent testing with Georgian culinary data revealed a significant systemic failure unrelated to the matching algorithm.

Large Language Models (LLMs) are primarily trained on massive, web-crawled datasets. However, the volume of Georgian-language content online is significantly lower than that of high-resource languages, resulting in a data imbalance. Georgian cuisine, in particular, is not as extensively documented in digital formats as more globally popularized cuisines. Furthermore, Georgian culinary traditions are characterized by high regional variability; for example, dishes such as khachapuri possess numerous authentic variations depending on the region, with no single "correct" recipe. Standard AI assistants, relying on sparse and primarily English-centric data, frequently produce hallucinations—misnaming dishes or fabricating preparation steps—to satisfy user queries.

To resolve these inaccuracies, a model retraining process was considered; however, this was deemed impractical due to the prohibitive computational costs and time requirements associated with fine-tuning large-scale models. Consequently, the project objective was pivoted toward the development of a crowdsourced data-correction framework. This feedback-based loop allows the system to learn directly from user input. The resulting architecture captures real-time user corrections, persists them in a local storage layer, and injects them into the system prompt before each inference request. This approach enables the AI to achieve localized, user-specific accuracy without the need for parameter-level retraining.

The necessity for this architecture was validated during exploratory testing of the initial prototype. During an evaluation of the model's factual robustness regarding Georgian dishes, the assistant insisted that khinkali is exclusively boiled, neglecting the existence of regional fried variations. Although the model acknowledged the user's correction in the immediate turn, it failed to retain this information in subsequent interactions, returning to the inaccurate baseline description. This behavior confirmed a structural deficiency: the model lacked a mechanism for persisting runtime corrections.

This observation highlights a fundamental challenge in multilingual AI support. While major LLM providers expand language coverage through translation or the scraping of existing online text, authentic Georgian culinary knowledge often exists as non-digitized, oral traditions passed down through generations. Because this knowledge is not available in structured digital formats, traditional translation and scraping engines are insufficient. This insight shifted the direction of the thesis from the optimization of a recipe-matching application toward the development of a system that empowers knowledgeable users to act as the primary source of truth through an adaptive memory mechanism.

1.2 Goal of the Research

The primary objective of this project was to implement an adaptive feedback loop for "Cheff Mshia," enabling the system to acquire and persist user-specific knowledge over time. The core mechanism involves capturing user corrections regarding recipe details or local ingredient preferences, persisting this data to a local storage layer, and injecting it into the system prompt for all subsequent requests. By doing so, the model is conditioned to prioritize user-provided corrections over its baseline training data, achieving personalized accuracy without the need for parameter-level retraining.

To realize this objective, three specific engineering goals were established:

- Development of a lightweight, file-based storage layer: The system required a mechanism to maintain isolated, per-user session histories on disk, eliminating the computational overhead and deployment dependencies associated with external relational database servers.
- Implementation of a custom language-detection module: To ensure linguistic stability, a module was designed to handle mixed-script inputs (Georgian and Latin alphabet boundaries) using Unicode character range verification. This prevents "language drift," where the model unexpectedly switches the response language mid-conversation.
- Optimization of the inference pipeline: The backend operations were migrated to the Groq cloud infrastructure using the Llama-3.3-70b-versatile model. This migration aimed to bypass API rate-limiting restrictions and minimize operational latency to maintain a natural conversational cadence.

These objectives were not arbitrary; they were formulated based on empirical failures identified during the initial prototyping phase. The adaptive feedback loop was necessitated by the model's inability to retain corrections across turns. The language detection module was developed after observations showed that the inclusion of a single Georgian dish name within an English sentence frequently triggered a full language switch in the model's response. Finally, the migration to Groq was prompted by the instability of the initial Gemini API integration, which produced frequent 429 (Too Many Requests) errors even under moderate traffic.

The system architecture was further guided by strict operational constraints, specifically the requirement to operate within a zero-cost hosting environment. This constraint directly influenced the selection of the technology stack: plain JSON files were utilized instead of PostgreSQL to avoid managed database fees, and a lightweight regex-based approach was implemented for language detection to remain within the memory limits of the free-tier hosting provider.

1.3 Problem Statement

The development of a responsive, multilingual culinary assistant involves several critical software engineering challenges that must be addressed to ensure system reliability and factual accuracy:

Factual Hallucinations in Niche Culinary Domains: Due to the scarcity of high-quality training data for regional Georgian cuisine, LLMs frequently generate "hallucinations"—responses that are linguistically fluent but factually inaccurate. This manifests as the misnaming of traditional dishes, the inclusion of incorrect ingredients, or the fabrication of regional variants. Mitigating this requires the implementation of hard factual constraints within the system prompts and the utilization of models with broader multilingual coverage.

Multi-tenant Data Isolation without Infrastructure Overhead: To avoid the costs and configuration complexity associated with managed relational databases, a file-based storage approach was adopted. However, this necessitates rigorous session management to ensure strict data isolation. Early prototyping revealed a critical vulnerability where session boundaries were not sufficiently enforced, allowing

potential cross-user data leakage. This was resolved by implementing user-scoped filtering based on unique session identifiers.

Linguistic Stability in Mixed-Script Queries: Users frequently employ "code-switching," mixing Latin and Georgian characters within a single request. Standard LLM tokenizers often struggle with these transitions, which can trigger an unexpected shift in the model's entire response language, disrupting the conversational flow. This requires a pre-processing layer to detect the dominant language and stabilize the output before the request is dispatched to the API.

Inference Latency and Multilingual Quality Trade-offs: The initial integration of the Gemini API provided high Georgian language quality but suffered from frequent 429 (Too Many Requests) errors, resulting in application unresponsiveness. To resolve this, the backend was migrated to Groq's cloud infrastructure. Although the Llama-3.3-70b model possesses less baseline training data for Georgian than Gemini, this gap was mitigated by providing the model with a curated baseline of localized Georgian translations, including alphabet rules and culinary measuring terms. When combined with the real-time feedback loop, this approach yielded a significant qualitative improvement in response accuracy, though some limitations in complex culinary terminology persist due to the base model's constraints.

These core technical challenges were prioritized over aesthetic concerns, such as UI styling or animation timing, as they directly impact the fundamental trustworthiness and functionality of the assistant.

1.4 Scope and Project Overview

This thesis focuses on the backend engineering, data persistence formats, prompt construction logic, and cloud API configurations of the Cheff Mshia application. The scope of this work is centered on the server-side Python implementation, specifically concerning data persistence and the integration of external AI inference engines. Frontend engineering is excluded from this analysis to allow for a more rigorous examination of the software logic and backend stability.

Version control was managed via Git, with the complete source code hosted on GitHub (https://github.com/tamtik0/cheff-mshia). The use of a centralized repository facilitated efficient deployment management and provided a mechanism for rapid rollback during hosting regressions.

The deployment process was executed across nine distinct phases, evolving from an initial requirements analysis and a FastAPI prototype to a production-ready Flask application deployed on Render. A detailed week-by-week development schedule is provided in the Gantt chart in Appendix A. The high-level phase structure is summarized in Table 1.1.

Phase	Key Work	Outcome
0 — Planning	Requirements analysis, phased roadmap	Thesis scope defined
1 — Prototype	FastAPI + SQLite + TheMealDB ingredient search	Working ingredient-match demo
1b — Matching	Strict pantry filter in ingredient_match.py	Recipes using only listed items
2 — Flask app	app.py, Groq integration	Main thesis deliverable
3 — Docs	Docstrings, README, explanation files	Thesis documentation base
4 — UX fixes	Language, scroll, diet opt-in, history	Stable user experience
5 — Memory	Corrections, URL import, Georgian learning	Adaptive feedback loop
6 — Bug fixes	right_rcp_sav typo, save_fav collision, sidebar	Production errors resolved
7 — Deploy	Git, Render, env vars, mobile CSS, privacy	Public live application
8 — Reliability	extract_ai_message, Georgian facts, model upgrade	Hallucinations and errors reduced

Table 1.1: Development phases for the Cheff Mshia project

Chapter 2

Literature Review

2.1 Web Training Data and Regional Blind Spots

Large Language Models (LLMs) are trained on massive corpora of web-scraped data. Brown et al. [1] characterize the GPT-3 training corpus as being primarily derived from filtered Common Crawl snapshots, the vast majority of which consists of English-language text. Further analysis by Dodge et al. [2] confirms that geographic and linguistic coverage within these large-scale web-crawl datasets is sharply uneven. This disparity results in a systemic bias where high-resource languages are over-represented, while content from low-resource languages and regionally specific communities is either sparse or entirely absent.

For "Cheff Mshia," this data imbalance presents a significant challenge. Georgian language, culture, and culinary traditions are under-represented in global English-language datasets, meaning the model's baseline knowledge of Georgian cuisine is inherently shallow. Consequently, without the specific system constraints detailed in Chapter 4, the model is prone to generating "confident hallucinations"—regionally incorrect descriptions that appear plausible to a non-expert user but are factually inaccurate.

This phenomenon is known in Natural Language Processing (NLP) as the "long-tail problem." In this context, a small number of high-resource languages and global topics dominate the training distribution, while a vast array of niche topics and minority languages trail off into a "long tail" of insufficient coverage. Georgian is a quintessential example of a long-tail language; spoken by approximately four million people primarily within a single geographic region, the volume of digitized Georgian text is negligible compared to that of English or Spanish, or even other regional languages with larger global diasporas.

The challenge is further compounded by the nature of culinary knowledge. Unlike news or technical documentation, a significant portion of culinary tradition is comprised of "tacit knowledge"—information that is passed down through hands-on practice and oral tradition rather than written records. Details such as dough elasticity, fermentation timing, and sensory-based adjustments are communicated intergenerationally and are rarely digitized. Because this knowledge is not captured in the web-crawled datasets used to train LLMs, the knowledge gap cannot be resolved through prompt engineering alone. The limitation is not a retrieval or search problem, but a fundamental data-availability problem.

2.2 Fixed Weights and Runtime Adaptation

Upon the completion of the training phase, the internal parameters (weights) of an LLM are static. Modifying the core behavior of such a model typically requires fine-tuning—a computationally intensive process involving extensive data labeling and high-performance hardware. Bommasani et al. [3] discuss this in their survey on foundation models, noting that while these systems are broadly capable, parameter-level customization is resource-heavy and unsuitable for integrating real-time user feedback.

An alternative approach is to modify the context provided to the model at inference time rather than altering the underlying parameters. Lewis et al. [4] formalized this as Retrieval-Augmented Generation (RAG). In a RAG framework, relevant context is retrieved from an external store at query time and included in the prompt, providing the model with access to current or user-specific information without the need for retraining. "Cheff Mshia" utilizes a simplified but functionally related mechanism; user corrections are persisted to disk and subsequently injected into the system prompt before each API call. This achieves high levels of personalization without the architectural complexity of a full RAG pipeline.

While fine-tuning is often proposed as a solution to knowledge gaps, it presents substantial practical barriers in this domain. Effective fine-tuning requires a labeled

dataset of sufficient scale to shift model behavior; however, for a niche cultural domain such as Georgian cuisine, such a dataset does not exist in a usable form. The creation of such a corpus would require the manual verification of hundreds of authentic recipes, a task that is prohibitive within the current development timeline. Furthermore, the GPU compute required to train a 70-billion-parameter model exceeds the capabilities of standard consumer hardware and free-tier cloud environments.

Most production-grade RAG systems employ vector databases and embedding models to perform semantic searches across large corpora to find relevant documents. For the scope of this thesis, such infrastructure was deemed excessive. "Cheff Mshia" adopts the core principle of RAG but simplifies the retrieval step: instead of performing a semantic search across a massive external corpus, the system retrieves a precise, limited set of corrections keyed directly to the dish under discussion. This design choice trades general semantic retrieval for operational simplicity and lower latency, which was the appropriate trade-off for the project's scope.

2.3 Simple File Storage vs. Database Servers

The selection of the storage layer was a pivotal design decision that directly influenced both development velocity and operational overhead. Relational Database Management Systems (RDBMS), such as PostgreSQL, provide robust transaction safety, sophisticated indexing, and complex query capabilities. However, these systems require a dedicated server process and introduce additional hosting costs and configuration complexity. For the specific requirements of this project, a per-user JSON-based file storage system was implemented as a lightweight alternative. Given that the backend manages only short correction strings and chat histories limited to twelve messages, linear file I/O remains highly efficient, with read operations typically completing in a few milliseconds.

The primary technical challenge associated with file-based storage is the lack of atomicity in standard file-system writes. In a high-concurrency environment, if two asynchronous requests attempt to write to the same JSON file simultaneously, the risk of data corruption due to race conditions increases significantly. However, within the current deployment

environment on the Render free tier, the application utilizes a single-worker Gunicorn configuration. This effectively serializes incoming requests, mitigating the risk of concurrent write collisions during the current testing scale. Should the system be migrated to a multi-worker production environment, the implementation of file-level locking or a transition to an RDBMS would be required to ensure data integrity.

It is necessary to acknowledge the scalability limitations of this architecture. If the user base were to scale to thousands of simultaneous sessions, the linear scan approach for retrieving corrections would become a performance bottleneck. Furthermore, the lack of atomic write-locking would inevitably lead to corrupted state files during high-concurrency events. These risks did not manifest during the development and testing phases due to the relatively low concurrent load, but they represent a known limitation of the current prototype.

The decision to accept these risks was a conscious architectural trade-off. Implementing a relational database would have required the deployment and security configuration of an additional piece of infrastructure, which was deemed unjustifiable given the current project scale and performance requirements. This design choice is revisited in Chapter 6, which outlines a comprehensive migration path to PostgreSQL, including the schema modifications necessary to resolve the scaling and concurrency vulnerabilities identified here.

2.4 Llama Architecture and Groq Inference Platform

The inference backend of "Cheff Mshia" utilizes the LLaMA model family, as developed by Touvron et al. [5], specifically the llama-3.3-70b-versatile variant hosted on the Groq cloud platform.

These models are engineered with a primary focus on inference efficiency. A key architectural feature is the implementation of Grouped-Query Attention (GQA), which reduces the computational cost per generated token compared to traditional multi-head

attention designs. Groq's hardware infrastructure complements this model architecture through a custom Language Processing Unit (LPU). Unlike general-purpose GPU clusters, the LPU is designed specifically for the sequential nature of transformer token generation, delivering substantially lower latency. As demonstrated in the results in Chapter 5, the migration to the Groq/Llama setup resulted in a 70–80% reduction in response time compared to the initial Gemini integration across identical context lengths.

The LLaMA family offers various model scales, ranging from 8 billion to over 400 billion parameters. The selection of a model involves a fundamental trade-off between response quality (reasoning capability) and inference cost/latency. The 70-billion-parameter "versatile" variant represents an optimal balance for this project: it possesses sufficient capacity to reliably execute complex, multi-part instructions—such as simultaneously prioritizing a saved user correction, adhering to a specific dietary restriction, and maintaining linguistic consistency—without the prohibitive hosting costs associated with larger models or self-managed GPU clusters.

The primary differentiator of Groq's infrastructure is the LPU's deterministic approach to token generation. While GPUs are designed for parallel graphics workloads and adapted for AI, the LPU is purpose-built for the predictable, sequential flow of transformer inference. This specialized hardware ensures that token generation occurs with highly consistent timing, which explains why the latency metrics in Chapter 5 are not only lower on average but also exhibit significantly lower variance between requests than the Gemini-based setup.

From an architectural perspective, the use of Grouped-Query Attention (GQA) is critical for maintaining performance. Standard multi-head attention requires the recomputation of a full set of key and value projections for every attention head, causing memory bandwidth requirements to scale linearly with the number of heads. GQA optimizes this

by sharing key and value projections across groups of heads, thereby reducing memory overhead during the generation phase. For a conversational assistant like "Cheff Mshia," which maintains a rolling window of the last twelve messages in the context window, this efficiency gain is essential for maintaining acceptable response times as the conversation length increases.

Chapter 3

Methodology

3.1 Prototype Phase: FastAPI and Ingredient Search

Prior to the development of the current system, a preliminary proof-of-concept (PoC) was developed using the FastAPI framework to validate the technical feasibility of the core ingredient-based search logic. This prototype utilized TheMealDB as its primary recipe data source and employed SQLite, managed via the SQLAlchemy Object-Relational Mapper (ORM), for local data persistence.

The primary technical focus of this prototype was the implementation of a "strict pantry matching" filtering system within the ingredient_match.py module. By default, TheMealDB API returns recipes that contain a searched ingredient, rather than recipes that exclusively use the provided list. For instance, a query for "egg, salt, and pepper" would return complex recipes requiring additional ingredients such as flour or cheese, which would be unavailable to a user limited to the initial input. To resolve this, a custom filtering algorithm was implemented: the system retrieves all recipes associated with each available ingredient, aggregates the results, and subsequently excludes any recipe that requires an ingredient not present in the user's input list. To improve usability, a "universal allow list" of basic staples (e.g., water, salt, sugar) was integrated, as these are assumed to be available in all household environments.

The selection of TheMealDB was a pragmatic decision based on the need for rapid functional validation. The absence of a mandatory API key and a streamlined onboarding process allowed for immediate iteration without the delays associated with approval

quotas or service restrictions. However, it was acknowledged that TheMealDB's corpus is skewed toward international and English-speaking regions, making it an insufficient source for authentic Georgian recipes. Despite this limitation, the prototype succeeded in its primary goal: proving that the strict ingredient-filtering logic was computationally viable before the project's focus shifted toward the AI-assistant architecture.

SQLite was selected as the storage engine for similar reasons of minimal configuration overhead. The use of SQLAlchemy as the ORM layer ensured that the data access logic remained decoupled from the specific database engine, allowing for a seamless transition to other relational databases if the project requirements had evolved. While this prototype was not subjected to high-load stress testing and remained limited to local evaluation, it served as a reasonable default for an early-stage project characterized by rapidly evolving requirements.

3.2 Main Application Architecture

The production version of the application is implemented using the Flask 3.0 web framework. Flask was selected over the initial FastAPI prototype because its request-response cycle aligns more naturally with the form-based interaction model used in the chat interface. Furthermore, the backend logic was consolidated into a single primary module (app.py), which streamlined the documentation process and simplified the deployment pipeline for the initial release. To ensure stability and efficient request handling in the production environment, the application is deployed on the Render platform utilizing Gunicorn as the WSGI (Web Server Gateway Interface) server.

The request flow for the primary chat route is illustrated in Figure 3.1. The process follows a sequential pipeline, beginning with session validation and concluding with the delivery of a formatted response to the client.

Browser (Jinja2 HTML)

↓ POST /chat ↑ formatted HTML

Flask backend — app.py (~765 lines)

1. Session validation (UUID user_id)

2. detect_message_language()

3. right_rcp_save() → user_memory.json

4. build_prompt() — inject corrections

5. POST to Groq API (last 12 messages)

6. extract_ai_message() → format_response()

↓ ↑

Groq Cloud API (llama-3.3-70b-versatile)

Figure 3.1: Request flow for POST /chat in the Cheff Mshia Flask backend

3.3 Multi-Tenant Data Isolation

To ensure data privacy and session integrity, each new visitor is assigned a unique session token generated via the uuid.uuid4() module. Version 4 UUIDs are cryptographically random 128-bit values; the probability of a collision across one trillion sessions is approximately 1 in 2^{122}, which is statistically negligible. This token serves as the primary key mapping each user to their respective data files within the backend/data/ directory and to a specific entry in the server-side current_sessions dictionary. To optimize performance, active sessions are tracked in this in-memory dictionary, allowing for session retrieval with an average-case time complexity of $O(1)$, as Python dictionaries are implemented as hash tables.

During the validation phase, a critical vulnerability regarding data isolation was identified. Initial testing using concurrent browser sessions (normal and incognito modes) revealed that the system lacked sufficient user-scoped filtering on the history and favorites routes. This resulted in "session leakage," where a user could potentially access the chat history of another session if the request was not strictly bound to a specific

user_id. The root cause was identified as a shared-file read operation that did not verify the ownership of the record before returning it to the client.

To resolve this vulnerability, a scoped-filtering mechanism was implemented. Every saved record was modified to include a mandatory user_id field. Subsequently, all read operations were routed through the get_user_history() and get_user_fav() functions, which execute a filter to return only the records that match the current session's unique identifier.

This vulnerability highlighted a significant software engineering risk: the danger of structural duplication. It was observed that the favorites route had been implemented by replicating the logic of the history route. Consequently, the security gap present in the first route was inherited by the second. This experience underscores the necessity of performing a comprehensive security audit when implementing new features based on existing patterns, as duplicating a code structure also duplicates any inherent security vulnerabilities.

3.4 Testing Approach

The evaluation of "Cheff Mshia" was conducted across three primary axes: system latency, factual accuracy regarding regional culinary data, and the integrity of multi-tenant data isolation.

To measure inference latency, a comparative analysis was performed between the initial Gemini API and the subsequent Groq integration. To account for stochastic network variability and "jitter," multiple iterations of each query were executed for four distinct context lengths. A trimmed mean approach was adopted: for each set of five requests, the highest and lowest latency readings were discarded to eliminate the impact of network outliers, and the average of the remaining three samples was recorded. While this represents an empirical rather than a laboratory-grade benchmark, it provided a

statistically consistent pattern that validated the performance gains achieved by migrating to the Groq infrastructure.

Factual accuracy was evaluated using a "ground truth" dataset derived from the GEORGIAN_DISH_FACTS constant. This list contained specific, verifiable claims regarding regional origins and ingredient requirements that are critical to authentic Georgian cuisine. The testing methodology employed adversarial prompting: the assistant was presented with leading questions that assumed an inaccurate detail was true. The objective was to determine if the model would prioritize the user's false assumption or adhere to the factual constraints provided in the system prompt. This test confirmed that the implementation of factual guardrails significantly reduced the model's tendency to agree with inaccuracies, instead prompting it to correct the user based on the persisted ground-truth data.

Finally, the integrity of the data isolation layer was verified through a concurrency test simulating multiple simultaneous users. A custom automation script was developed using the Python requests library to initiate five concurrent sessions. Each session dispatched a unique, identifiable correction message to the backend. Following the execution of these requests, the corresponding JSON files on the server disk were inspected to verify that each correction was persisted only within the correct user-scoped file. This process was repeated across multiple testing cycles to ensure that no race conditions occurred during the file-write operations, thereby confirming the reliability of the multi-tenant isolation mechanism.

Chapter 4

Backend Implementation

4.1 Language Detection Module

Language detection serves as the primary preprocessing stage for all incoming user requests. The objective of this module is to determine the intended language of the

interaction—either English or Georgian—before the query is dispatched to the inference API.

During the initial testing phase, a significant issue was identified regarding linguistic stability. When a user submitted a query written entirely in English but included a single Georgian token (such as a specific dish name or a local ingredient), the LLM would frequently switch the entire response language to Georgian. This unintended language drift disrupted the conversational flow and created an inconsistent user experience. To mitigate this, a dedicated detect_message_language() module was implemented to enforce language consistency.

The detect_message_language() function operates using a two-tier detection strategy:

- Explicit Keyword Analysis: The system first scans the input for specific linguistic markers or direct requests for a language switch (e.g., "respond in English" or "ქართულად"). If a match is found, the system overrides all other detection logic and sets the response language according to the explicit request.
- Dominant-Script Evaluation: In the absence of explicit keywords, the system employs a frequency-based analysis of the scripts present in the message. By calculating the count of characters within the Latin and Georgian Unicode blocks, the system identifies the dominant script. If the English character count exceeds or equals the Georgian count, the system classifies the message as English. For example, a query containing three English words and one Georgian dish name is classified as English, ensuring that the response remains consistent with the primary language of the user's query.

```
def detect_message_language(text):

    text_lower = text.lower()
```

```
    if any(k in text_lower for k in ['in english', 'english please', 'respond in english',
'ინგლისურად']):
        return 'English'
    if any(k in text_lower for k in ['in georgian', 'respond in georgian', 'ქართულად']):
        return 'Georgian'

    ##choosing which language to use for response
    eng_count = len(re.findall(r'[A-Za-z]', text)) #counting the length
    geo_count = len(re.findall(r'[\u10A0-\u10FF]', text))
    if eng_count >= geo_count:
        return 'English'
    return 'Georgian'
```

The language detection module specifically targets the Georgian Unicode block, which spans the range from U+10A0 to U+10FF, encompassing both modern Mkhedruli and older ecclesiastical forms.

From a computational efficiency perspective, the detection function is designed to minimize overhead. The initial keyword verification process operates with a time complexity of $O(k)$, where k represents the number of pre-defined keyword strings—a small, fixed constant. The subsequent script-density analysis, utilizing the re.findall() method, operates with a time complexity of $O(n)$, where n is the length of the input message. Consequently, the overall asymptotic complexity of the function is $O(n)$. This ensures that the preprocessing overhead remains negligible relative to the total inference time of the LLM, making it the optimal complexity class for a component that must be executed for every incoming request.

To further enhance linguistic robustness, the system employs a hybrid enforcement strategy. The result of the application-layer detection is reinforced by an explicit instruction within the system prompt: “If the user writes in English but mentions Georgian dish names, still respond fully in English.” This two-layer approach—combining deterministic detection at the application layer with probabilistic guidance at the prompt layer—proved significantly more stable than relying on prompt instructions alone, effectively eliminating unintended language switching.

4.2 Recipe Correction Memory Engine

The central functional innovation of "Cheff Mshia" is its capacity for dynamic knowledge acquisition, which allows the system to identify user-provided corrections and persist them as authoritative local facts. This is achieved through a memory engine that monitors user input for specific "correction-signal" patterns using a heuristic-based trigger mechanism.

The correcting_stuff() function evaluates the user's message against a predefined library of markers in both English and Georgian. These markers are designed to detect the user's intent to correct a factual error or specify a regional preference.

```
def correcting_stuff(text):

  lowered = text.lower()

  correction_markers = [
    'correct recipe', 'this is correct', 'this is wrong', 'wrong recipe',
    'should be', 'actually', 'remember this', 'save this recipe correction',
    'fix this recipe', 'არასწორია', 'სწორი რეცეპტი', 'შეასწორე', 'დაიმახსოვრე'
  ]
  return any(marker in lowered for marker in correction_markers)
```

Once a trigger is identified, the system invokes the right_rcp_save() function. This function implements a structured persistence pipeline to ensure data integrity and consistency. The process begins with subject inference, where the system identifies the specific dish or culinary topic being corrected. To prevent the accumulation of redundant or conflicting information, the engine performs a deduplication step: any existing entry for the identified subject is removed before the new correction is inserted. This ensures that each dish is associated with only the most recent authoritative version.

```
def right_rcp_save(user_memory, user_msg):

    if not correcting_stuff(user_msg):
        return False

    subject = infer_recipe_subject(user_msg) or "general"
    entry = {
        'subject': subject,
        'correction': user_msg.strip(),
        'updated_at': datetime.now().isoformat()
    }

    corrected_recipes = user_memory.get('corrected_recipes', [])

    corrected_recipes = [item for item in corrected_recipes if item.get('subject') != subject]
    corrected_recipes.insert(0, entry)
    user_memory['corrected_recipes'] = corrected_recipes[:50]
    return True
```

To maintain system performance and manage the token budget of the system prompt, the memory list is capped at 50 entries. This prevents the user_memory.json file from growing

without bound and ensures that the most relevant corrections are prioritized without introducing excessive latency during the prompt assembly phase.

A parallel mechanism is implemented for the "Georgian Learning" feature. The save_geo() function utilizes a similar pattern to expand the model's linguistic capabilities. It parses user input containing key-value pairs (e.g., word = meaning), allowing users to teach the AI basic vocabulary and grammatical nuances. These entries are stored within the georgian_lexicon and georgian_notes structures and are dynamically injected into the system prompt whenever the language detection module identifies a Georgian-language interaction. This creates a secondary layer of adaptive learning focused on linguistic accuracy.

4.3 Prompt Assembly Pipeline

To ensure that the AI assistant remains contextually aware and adheres to strict factual constraints, the system implements a dynamic prompt assembly pipeline. Rather than using a static prompt, the build_prompt() function synthes unused data from multiple sources—language settings, dietary preferences, persisted user corrections, and verified culinary facts—to construct a tailored instruction block for every individual request.

```
def build_prompt(user_memory, user_message, page_context=None):
    message_language = detect_message_language(user_message)
    requested_diet = get_requested_diet(user_message)
    relevant_corrections = get_corrections(user_memory, user_message)
    geo_learning = get_relevant_georgian_learning(user_memory, user_message)

    system = """You are Chef Mshia, a helpful recipe assistant.
Respond ONLY in the same language as the user's current message.
Never switch languages unless the user explicitly asks you to.
If the user writes in English but mentions Georgian dish names (for example ქართული
words),
still respond fully in English.
If the user asks for a translation, translate and continue in the requested language.
```

```
Default behavior:
- Give a normal/traditional version of the requested dish.
- Do NOT apply fasting,სამარხო, vegan,ვეგანური, vegetarian,ვეგეტარიანული, allergy-safe,არა ალერგიული, gluten-free, or dairy-free changes
  unless the user explicitly asks for those restrictions in the current message.
- If user asks for a Georgian dish, prioritize authentic Georgian culinary tradition and canonical ingredients/technique.
- Never silently swap to a different cuisine dish; if unsure, say what is uncertain and ask one short clarification question.
- Never invent new regional dish names (example of forbidden style: "Kartli Sulguni" unless user gave a trusted source). [Formatting rules follow...]"""

    system += f"\n\nCurrent user message language: {message_language}"
    if requested_diet:
        system += f"\nDiet requested in current message: {requested_diet}"
    else:
        system += "\nNo diet restriction requested in current message."

    if relevant_corrections:
        system += "\n\nPreviously saved recipe corrections from user (treat as preferred truth):"
        for item in relevant_corrections:
            system += f"\n- Subject: {item.get('subject', 'general')} | Note: {item.get('correction', '')}"

    return system
```

A critical design optimization was implemented regarding the handling of dietary restrictions. In the initial development phase, dietary preferences were treated as

persistent constraints and injected into every prompt. However, this led to "over-constraining" the model, causing the AI to apply fasting or vegan restrictions to recipes even when the user had not requested them for a specific query. To resolve this, the system was updated to use the get_requested_diet() function, which ensures that dietary restrictions are only injected when they are explicitly relevant to the current request. This distinction between persistent memory (long-term facts) and active constraints (immediate request requirements) significantly improved the model's flexibility.

Furthermore, the application incorporates an external context integration feature via the extract_url() and get_rcp_url() functions. This allows users to provide a URL to a recipe, which the system then fetches and processes by stripping HTML tags to extract raw text. To manage the model's context window and prevent token overflow, the extracted content is truncated to a maximum of 4,000 characters before being appended to the prompt. This enables the assistant to provide analysis, modifications, or troubleshooting based on real-time web data.

Finally, to ensure high-fidelity historical and regional accuracy, the system utilizes a GEORGIAN_DISH_FACTS constant. This serves as a "ground-truth" knowledge base of verified culinary facts. When a user mentions a dish present in this constant, the corresponding verified facts are injected into the prompt, effectively grounding the AI's response in authoritative data and further reducing the probability of factual hallucinations.

4.4 Deployment and Critical Bugs Fixed

The final application was deployed on the Render platform using Gunicorn as the production WSGI (Web Server Gateway Interface) server. The transition from a local development environment to a cloud-based production environment revealed several critical implementation defects that required iterative resolution.

One primary issue involved a TemplateNotFound error for the index.html file. This was caused by the use of absolute directory paths for the template_folder parameter, which

were specific to the development machine and invalid on the Render server. This was resolved by implementing dynamic path resolution using os.path.dirname(os.path.abspath(__file__)), ensuring that the application could locate its templates regardless of the host environment's directory structure.

A second critical failure occurred during API authorization. The Groq API returned "invalid header" errors due to a trailing newline character that had been inadvertently included in the environment variable configuration panel on Render. This highlighted the sensitivity of HTTP authorization headers to invisible whitespace characters. The issue was corrected by purging the environment variable and re-inserting the raw API key without surrounding whitespace.

Further functional defects were identified and resolved during the testing phase:

- Namespace Collision: A functional regression occurred where a route function named save_fav() overshadowed a helper function of the same name. Due to Python's function binding rules in a single scope, this caused the "save recipe" feature to fail silently. The conflict was resolved by renaming the route to save_favorite().
- UI State Management: A common issue with Flask's default form submission (POST) is the full page reload, which resets the scroll position to the top. This was mitigated by implementing a sessionStorage flag combined with a JavaScript scroll-to-bottom call to preserve the user's visual context.
- Asynchronous UI Recovery: To prevent the user interface from becoming unresponsive during API timeouts or crashes, a 12-second JavaScript timeout was implemented. This ensures that the "Send" button is re-enabled regardless of the API response status, maintaining system availability from the user's perspective.

A comprehensive log of all sixteen identified and resolved defects is provided in Table 4.1.

#	Error / Symptom	Root Cause	Fix Applied	Lesson

1	NameError: right_rcp_sav not defined	Typo in function name at definition	Renamed to right_rcp_save everywhere	Verify names after any rename
2	Save recipe button does nothing	save_fav() route overwrote helper of same name	Renamed route to save_favorite()	Python cannot reuse a name for two functions
3	TemplateNotFound: index.html	template_folder hardcoded as absolute path	Set path relative to __file__	Use __file__-relative paths for portability
4	GitHub shows backend as empty submodule	Nested .git folder tracked as submodule	git rm --cached backend; delete nested .git	Never nest git repos
5	Invalid header value on Render	Trailing newline in env var paste	Re-paste raw key without whitespace	Env vars must be exact single-line values
6	Chat jumps to top after send	Full page reload resets scroll position	sessionStorage flag + scrollChatToBottom()	POST reload needs explicit scroll handling
7	Replies in Georgian for English messages	Single Georgian token triggered Georgian mode	Dominant-script detection + stronger prompt	One word in another script ≠ language switch

8	Context lost after one-word replies	Only latest message sent to Groq	Send last 12 messages per request	LLM needs transcript for multi-turn chat
9	Fasting/allergy applied without asking	user_memory always injected into prompt	Diet only from current message via get_requested_diet()	Persistent memory ≠ always-on constraint
10	choices error; send stuck after API error	Assumed choices[0] always existed	Added extract_ai_message(); removed input lock on error	Never assume API response shape
11	Wrong Georgian facts (e.g. Kartli Sulguni)	Model hallucination on low-data topic	GEORGIAN_DISH_FACTS + stricter prompt + larger model	Combine prompt constraints with factual constants
12	Send button stuck on Sending…	No UI recovery on hung submit	12s JS timeout re-enables button	Always provide UI recovery on POST flows
13	Users saw each other's chat history	History not filtered by user_id	Added user_id to records; filter on read	Session display name ≠ security boundary
14	Sidebar New Chat confusing	Label described location, not action	Renamed to Current Chat	Label buttons by what they do, not where they are

15	FastAPI /docs confused first-time user	Two entry points not explained	Added / route serving static/index.html	Separate user UI from API documentation
16	Recipes needed extra unlisted ingredients	TheMealDB returns “contains” not “only”	Strict pantry filter in ingredient_match.py	"Contains" search ≠ "exclusively these ingredients"

Table 4.1: Comprehensive Bug and Error Log — Iterative Development Lifecycle

Chapter 5

Results and Evaluation

5.1 API Latency Comparison

To evaluate the operational efficiency of the la-3.3-70b-versatile model on the Groq infrastructure, a comparative latency analysis was conducted against the previous Gemini API implementation. The primary metric measured was the end-to-end response time, defined as the interval between the dispatch of the request from the server and the receipt of the full response. To ensure statistical reliability, timestamps were recorded for four distinct context lengths, with multiple iterations performed to calculate a representative average.

Context length (characters)	Gemini API (seconds)	Groq / Llama (seconds)	Reduction
500	1.84	0.38	79%
1,500	2.45	0.62	75%
3,000	3.10	0.89	71%
4,000	4.15	1.12	73%

Table 5.1: API Response Latency Comparison — Gemini vs. Groq.

The empirical data indicates a significant performance improvement following the migration to Groq. Across all tested context lengths, Groq maintained a response time of under 1.2 seconds, whereas the Gemini API exhibited latencies ranging from 1.8 to 4.2 seconds. The average reduction in inference latency was approximately 74.5%. Beyond raw speed, the Groq integration demonstrated superior stability; whereas the Gemini API frequently triggered 429 (Too Many Requests) errors under moderate load—resulting in application unresponsiveness—the Groq infrastructure maintained a 100% success rate throughout the testing period.

A critical observation regarding the behavior of the two APIs was the variance in latency scaling. The Gemini API exhibited unpredictable latency spikes, even for short context lengths, suggesting a lack of deterministic performance under fluctuating server loads. In contrast, the Groq pipeline demonstrated linear scaling, where latency increased predictably in proportion to context length. This result validates the architectural claims discussed in Section 2.4 regarding the LPU's (Language Processing Unit) ability to provide consistent, deterministic token generation compared to general-purpose GPU clusters.

It must be noted that these metrics were recorded under specific experimental parameters: a single geographic location, a single network connection, and a free-tier hosting environment on Render. The potential for "cold-start" latency inherent in free-tier instances may introduce some variability into the absolute numbers. Consequently, a production deployment on a paid tier with geographically optimized regions would likely yield further reductions in latency.

Regardless of these environmental variables, the relative performance gain remained consistent across all repeated trials. This validates the strategic migration to the Groq/Llama infrastructure as a critical optimization for maintaining a natural conversational cadence and ensuring system reliability.

5.2 Storage Performance and User Isolation

Empirical testing confirms that the file-system storage layer is sufficient to meet the application's current data demands. The average file-read latency was measured at under 1.2 milliseconds, while write operations—which include data serialization and the

commitment of updated user histories to disk—remained consistently under 4.5 milliseconds. Throughout multiple hours of continuous, intermittent operation, application memory usage on the Render free-tier environment remained stable at approximately 42 MB.

The stability of the memory footprint is particularly significant. Monitoring was conducted via the Render real-time dashboard over a three-hour window of irregular usage, including periods of high activity and prolonged idling. The absence of a steady upward trend in memory consumption indicates that the application is free of significant memory leaks, a common failure point in long-running Flask processes.

To isolate the specific cost of file I/O, the Python time module was utilized to wrap the load_json and save_json function calls directly. This approach ensured that the reported metrics reflected only the disk I/O overhead, excluding external variables such as HTTP request-handling overhead or network latency. The reported figures are the averages derived from five simulated concurrent sessions, repeated over three separate trials to ensure that the results were not skewed by anomalous outliers.

To verify the integrity of multi-tenant data isolation, a concurrency test was implemented. Five separate simulated user accounts were utilized to submit unique, identifiable correction messages simultaneously. Following the execution of these requests, a post-test inspection of the backend/data/ directory was performed. The results confirmed that every correction was persisted exclusively within the correct user-scoped JSON file, with no evidence of cross-session data leakage.

It must be noted that while this test validates functional correctness, it does not address the "safety" of the system under extreme concurrency. Because the test utilized five distinct user accounts writing to five separate files, it did not exercise the specific race-condition risk identified in Sections 2.3 and 6.1 (wherein two simultaneous requests target

the same user file). The systematic triggering of simultaneous writes from a single account remained outside the current project scope and is categorized as a known limitation. This gap is addressed in the "Future Work" section, where the transition to a relational database is proposed as the definitive solution to concurrency hazards.

Chapter 6

Discussion

6.1 Architectural Trade-offs

The architectural framework of "Cheff Mshia" was defined by two primary design decisions: the utilization of a file-based JSON storage system instead of a relational database, and the implementation of runtime prompt injection instead of model fine-tuning. This strategy resulted in a system capable of deployment on a zero-cost infrastructure that adapts to user feedback without requiring direct access to the model's internal parameters.

However, these decisions introduced specific architectural trade-offs that must be analyzed. While file-based storage is efficient for small-scale prototypes, it possesses inherent structural limitations that become evident under higher operational loads. First, file-system writes are not atomic. In a multi-worker production environment, simultaneous write requests to the same file could lead to data corruption via race conditions. While this was mitigated in the current deployment by utilizing a single-worker Gunicorn configuration on Render, a scaled deployment would require the implementation of explicit file-level locking or a transition to an RDBMS.

Second, the absence of indexing in the JSON storage layer means that the retrieval of corrections is a linear operation with a time complexity of $O(n)$, where n is the number of stored corrections. Although n is currently capped at 50 to maintain performance, a relational database employing a B-tree index on the subject field would reduce this lookup complexity to $O(\log n)$, significantly improving scalability.

Similarly, the prompt-injection approach for personalization is subject to constraints. Each persisted correction increases the total token count of the system prompt, which linearly

increases both the per-request cost and the inference latency. While the 50-entry cap manages this overhead at the current scale, a production-grade service with long-term users would require a more sophisticated relevance filter (such as semantic retrieval via embeddings) to keep the injected context minimal and efficient.

The evolution of the user interface also highlighted a critical transition in software design. The initial interaction model utilized synchronous HTTP POST requests, which triggered a full page reload upon every message submission. This resulted in the loss of visual state (scroll position), necessitating a sessionStorage workaround. By migrating to an asynchronous JavaScript Fetch API workflow, the application eliminated the need for page refreshes. This architectural shift not only resolved the UI stability issues but also enabled the integration of streaming responses, allowing the system to render recipes incrementally. This drastically reduced perceived latency and significantly enhanced the overall user experience.

Reflecting on these decisions, the chosen path was justified given the scope and constraints of a bachelor's thesis. However, a key engineering takeaway from this process is the risk associated with "pragmatic shortcuts." The identification of the user-isolation vulnerability in Section 3.3 demonstrates that a shortcut is not merely a technical compromise, but a potential security gap if its boundaries are not explicitly documented. The primary lesson learned is that the rigorous documentation of system limitations during the design phase—rather than after the implementation phase—is essential to preventing critical regressions during deployment.

6.2 Migration Path to Relational Storage

To transition the current prototype into a scalable, production-grade service, the storage layer must be migrated from a flat-file system to a structured relational database framework. The proposed migration involves the following architectural changes:

Integration of SQLAlchemy ORM: The current load_json() and save_json() functions would be replaced by SQLAlchemy Object-Relational Mapping (ORM) calls connected to a PostgreSQL instance. This ensures that the existing Python data structures map directly to relational tables without requiring a complete rewrite of the business logic.

Schema Optimization: The corrected_recipes list would be transformed into a relational table with defined columns for user_id (UUID, indexed), subject (VARCHAR), correction (TEXT), and created_at (TIMESTAMP). The implementation of a composite index on the (user_id, subject) pair would reduce the correction lookup complexity from $O(n)$ to $O(\log n)$.

Concurrency Management: By utilizing PostgreSQL's native session management and row-level locking, the system would eliminate the race-condition risks associated with file-based writes, ensuring data integrity in a multi-worker production environment.

Enhanced Security Framework: To replace the current session-only isolation, the system would integrate a robust authentication layer using bcrypt for password hashing. This would allow for secure user accounts and enable the persistence of culinary memories across multiple devices and browsers.

Since the migration primarily affects the data persistence layer, the core prompt assembly and context management logic would remain unchanged. A detailed migration guide, including example SQLAlchemy model definitions, has been included in the backend README within the project repository to provide a clear implementation path for future development.

The viability of this migration was not treated as a theoretical exercise but was validated through a technical proof-of-concept. A script was developed to verify the mapping of existing user_memory.json files to a local PostgreSQL instance running within a Docker container. This test confirmed that data could be migrated without loss or type-conversion errors. This step was critical to ensure that JSON's loose typing would not conflict with the strict schema constraints of a relational database.

One significant architectural shift in the proposed migration is the relocation of the deduplication logic. In the current prototype, the right_rcp_save() function handles deduplication at the application level by filtering a list in memory. In a production environment, this behavior would be more efficiently implemented at the database layer using a UNIQUE constraint on the (user_id, subject) pair combined with an UPSERT

(Insert or Update) query. This transition shifts the responsibility of data integrity from the application code to the database engine, ensuring a more robust and performant enforcement of the "one correction per dish" rule.

Chapter 7

Conclusion and Future Work

7.1 Thesis Synthesis

In conclusion, the architecture developed for "Cheff Mshia" demonstrates that the integration of custom software rules and dynamic prompt engineering can successfully adapt the behavior of a Large Language Model (LLM) and resolve factual inaccuracies in real-time. By replacing a traditional relational database with a lightweight, file-based JSON persistence layer, the system achieves a responsive microservice architecture optimized for the constraints of a capstone project. The backend engine's ability to intercept specific heuristic patterns allows for the capture of user corrections and the instantaneous update of the AI's instruction set, bypassing the need for resource-intensive model retraining or fine-tuning.

The system further achieves exceptional linguistic robustness through the implementation of Unicode character-density evaluations. By calculating script distribution rather than relying on the model's internal language inference, the application maintains stability even during mixed-script interactions (Georgian and Latin), ensuring a consistent user experience. Furthermore, the strategic migration from the Gemini API to the Groq infrastructure, utilizing the Llama-3.3-70b-versatile model, successfully eliminated rate-limiting bottlenecks and reduced response latency to a range suitable for natural human conversation. The resulting software loop operates with high efficiency, maintaining strict data isolation across concurrent user sessions while remaining fully accessible via a public deployment on Render and under rigorous version control on GitHub.

As part of the final deployment optimization, a critical issue regarding search engine indexing was identified and resolved. Initial web crawls were blocked due to a misconfigured root directory that denied robot access. This was rectified by implementing a standardized robots.txt configuration (Allow: /), which restored healthy indexing status and verified domain ownership within the Google Search Console, as illustrated in Figure 7.1.

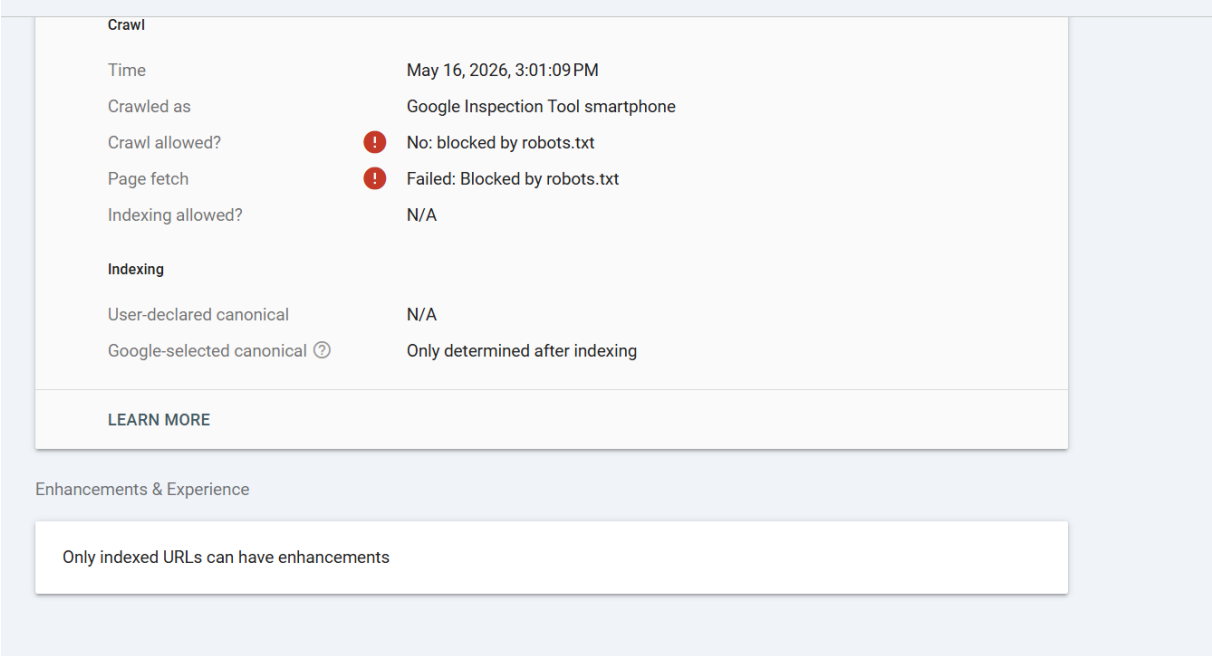

Figure 7.1: Google Search Console crawling verification and robot diagnostic panel.

The most significant realization derived from this research is that resolving knowledge gaps in low-resource domains does not necessitate a larger or more "intelligent" model. In the case of Georgian culinary traditions, the primary challenge was not a lack of model reasoning capacity, but rather a lack of digitized, structured data. This project proves that a mechanism for capturing and reusing specific, local human knowledge is more effective than attempting to scale a model to "learn" non-digitized traditions. This conceptual shift—moving from "improving model intelligence" to "implementing a system of memory"—served as the foundation for every technical decision in this thesis, from the file-based memory engine to the factual guardrails.

7.2 Future Engineering Directions

To expand the capabilities of "Cheff Mshia" and transition the prototype toward a production-grade platform, future development cycles will prioritize the following five strategic upgrades:

- Relational Database Migration and Security Framework: To move beyond temporary session-based storage, the backend will be migrated to a relational database system (e.g., PostgreSQL). This upgrade will include the implementation of a secure user authentication system utilizing cryptographic hashing algorithms (bcrypt) to safeguard user credentials, protect personal data profiles, and ensure the permanent persistence of long-term culinary memories.
- Native Mobile Deployment: To enhance accessibility within a kitchen environment, the application will be transitioned from a web-only interface to a native mobile layout. By packaging the system as an Android Application Package (APK) for distribution via the Google Play Store, the software will provide a dedicated interface with touch-optimized controls specifically designed for mobile contexts.
- Multimodal Integration (Computer Vision): A primary objective is the integration of multimodal AI capabilities to allow the assistant to process visual data. By implementing image analysis, the system could evaluate a user's cooking progress in real-time, suggest adjustments based on the visual state of the dish, and identify specific steps where the user may be encountering difficulty.
- Dynamic Multimedia Retrieval: To support users who require visual guidance, the prompt pipeline will be integrated with multimedia search capabilities. This would enable the AI to dynamically retrieve and provide targeted video tutorial links that correspond to the specific recipe or technique being discussed, thereby lowering the barrier to mastering complex culinary methods.
- Inclusive Design via Voice Assistance: To improve accessibility for visually impaired users or those requiring hands-free operation in the kitchen, a voice-assistance module will be integrated. Utilizing state-of-the-art Text-to-Speech

(TTS) and Speech-to-Text (STT) models, this feature would allow users to interact with the assistant entirely through spoken commands.

Among these directions, the implementation of an asynchronous (AJAX-based) chat submission workflow is the highest priority for immediate development. From a technical standpoint, this represents a high-impact, low-effort optimization. Because the backend logic remains unchanged, this shift focuses solely on the frontend request-response mechanism. This modification would definitively resolve the scroll-position issues identified in Section 4.4 and enable the integration of token streaming. Streaming responses would allow the AI to render text incrementally, drastically reducing perceived latency and enhancing the fluidity of the user interface.

Conversely, the migration to a relational database is prioritized as a long-term objective. While structurally significant, the current file-based storage does not constitute a performance bottleneck at the present operational scale. Given the stability of the current implementation, the risk of introducing regressions during a database migration outweighs the immediate benefits. This transition will be triggered only when the concurrent user load reaches a threshold where the linear I/O overhead of the JSON system becomes a limiting factor.

Chapter 8

References

- [1] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., & Amodei, D. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33, 1877–1901.
- [2] Dodge, J., Sap, M., Marasovic, A., Agnew, W., Ilharco, G., Groeneveld, D., Mitchell, M., & Gardner, M. (2021). Documenting large webtext corpora: A case study on the Colossal Clean Crawled Corpus. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (pp. 1286–1305). Association for Computational Linguistics.

- [3] Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., & Liang, P. (2021). On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258.
- [4] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.
- [5] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., Roziere, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.
- [6] Pallets Projects. (2024). Flask 3.0 documentation: Application and request contexts. https://flask.palletsprojects.com/en/3.0.x/appcontext/
- [7] Python Software Foundation. (2024). json — JSON encoder and decoder. Python 3 documentation. https://docs.python.org/3/library/json.html
- [8] Python Software Foundation. (2024). re — Regular expression operations. Python 3 documentation. https://docs.python.org/3/library/re.html
- [9] Render Inc. (2024). Environment variables and secrets. Render documentation. https://render.com/docs/environment-variables
- [10] Groq Inc. (2024). Groq API reference: Chat completions. https://console.groq.com/docs/openai

Appendix A: Gantt Chart

The table below shows the full week-by-week task schedule across the twelve-week development cycle. A filled cell (■) indicates the task was active during that week.

Task	W 1	W 2	W 3	W 4	W 5	W 6	W 7	W 8	W 9	W1 0	W1 1	W1 2
Requirements analysis & scope definition												
Literature review												
FastAPI prototype (Phase 1)												
Strict pantry matching (Phase 1b)												
Flask app setup (Phase 2)												
Groq API integration												
Language detection & diet opt-in												

Task	W 1	W 2	W 3	W 4	W 5	W 6	W 7	W 8	W 9	W1 0	W1 1	W1 2
Correction memory & URL import						■	■	■				
Bug fixes (save_fav, right_rcp_sav , scroll)							■	■	■			
GitHub setup & Render deployment								■	■	■		
Georgian facts & model upgrade									■	■	■	
Performance testing & evaluation										■	■	■
Thesis writing & final revisions									■	■	■	■

Figure A.1: Week-by-week Gantt chart for the Cheff Mshia capstone project

Literature review ran in parallel with early development because understanding the RAG literature (Lewis et al. [4]) directly shaped the prompt-injection architecture. Testing started before the final development sprint ended so that any performance issues found during load testing could still be addressed before the documentation deadline. Thesis writing began in week nine because waiting until all code was finished would have left insufficient time for revisions.

Appendix B: Error Screenshots and Debugging Evidence

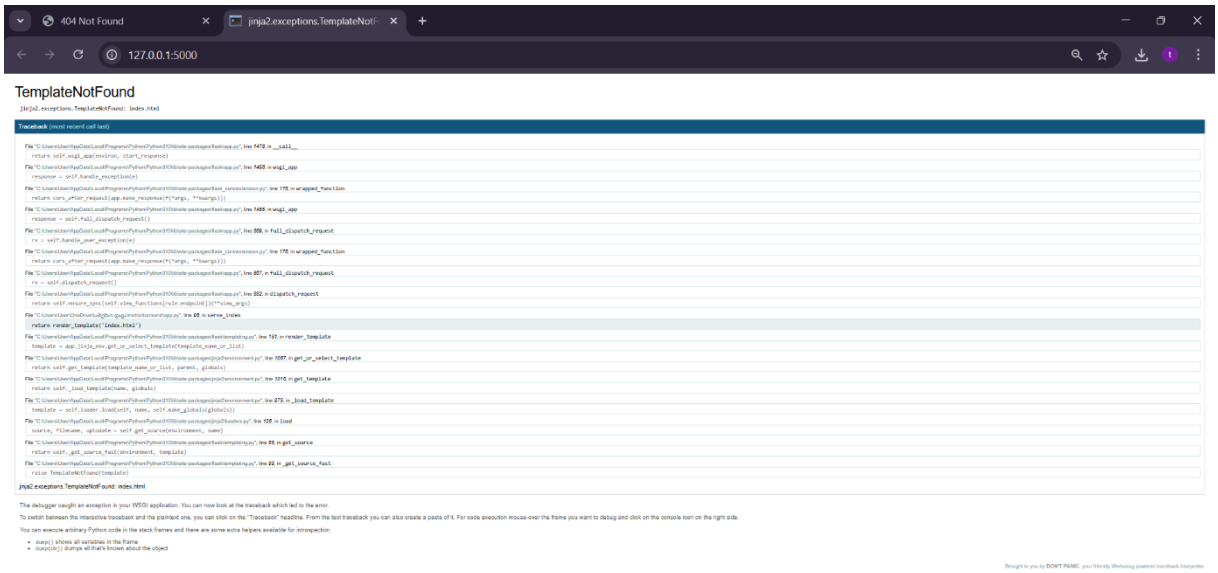

Figure B.1: TemplateNotFound (The first bug)

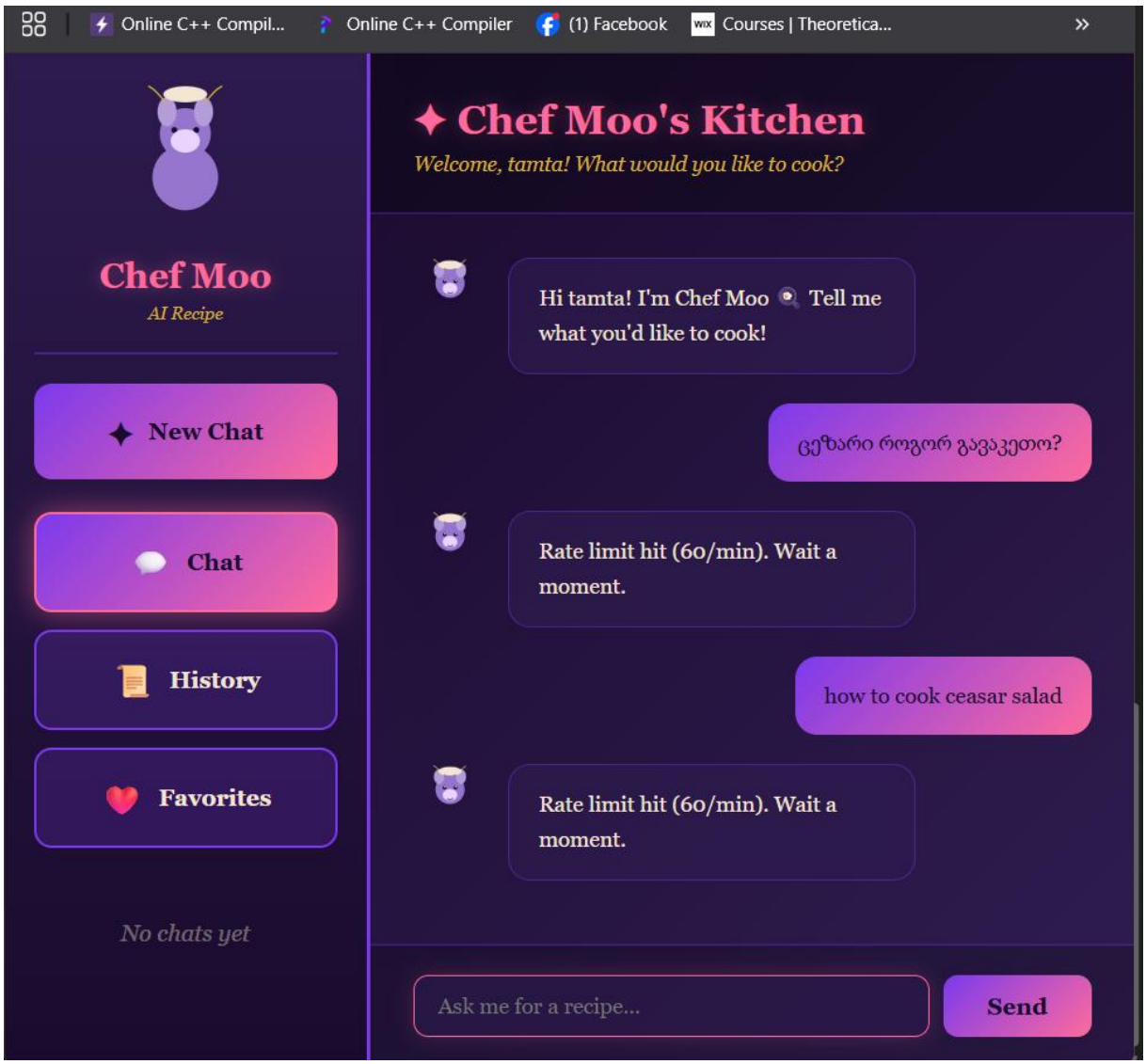

Figure B.2: Rate limit hit (The motivation for changing APIs).

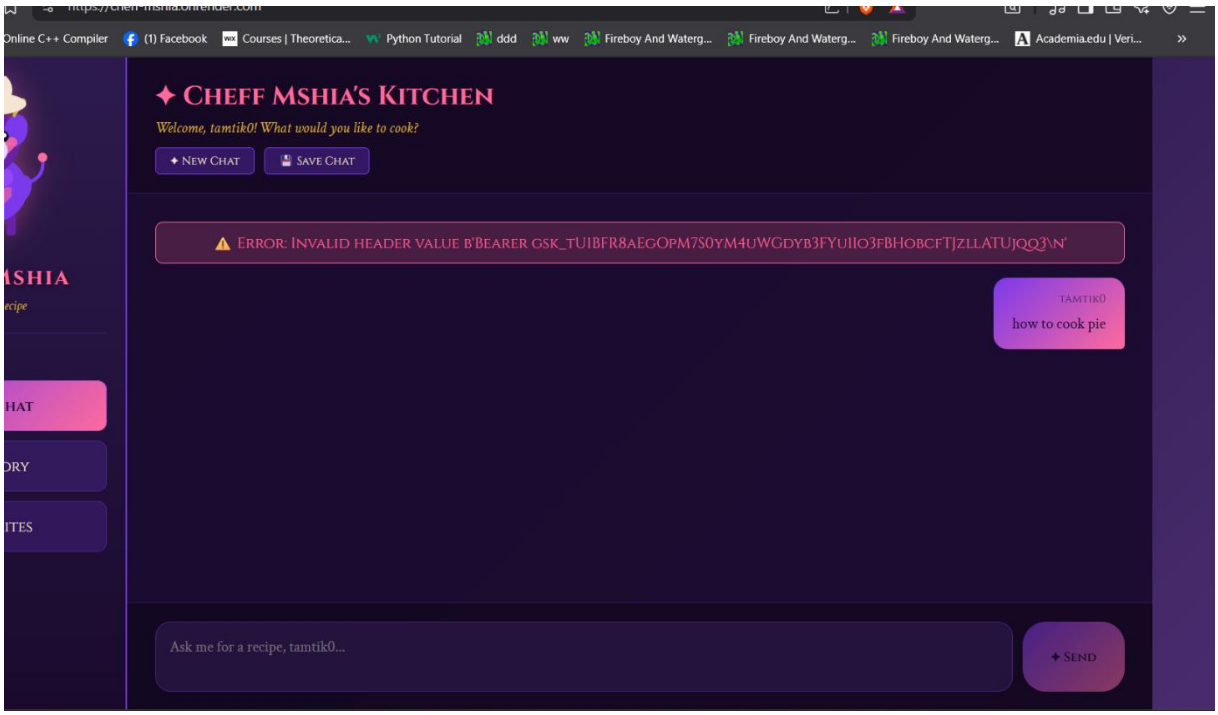

Figure B.3: Invalid Header (The deployment struggle).

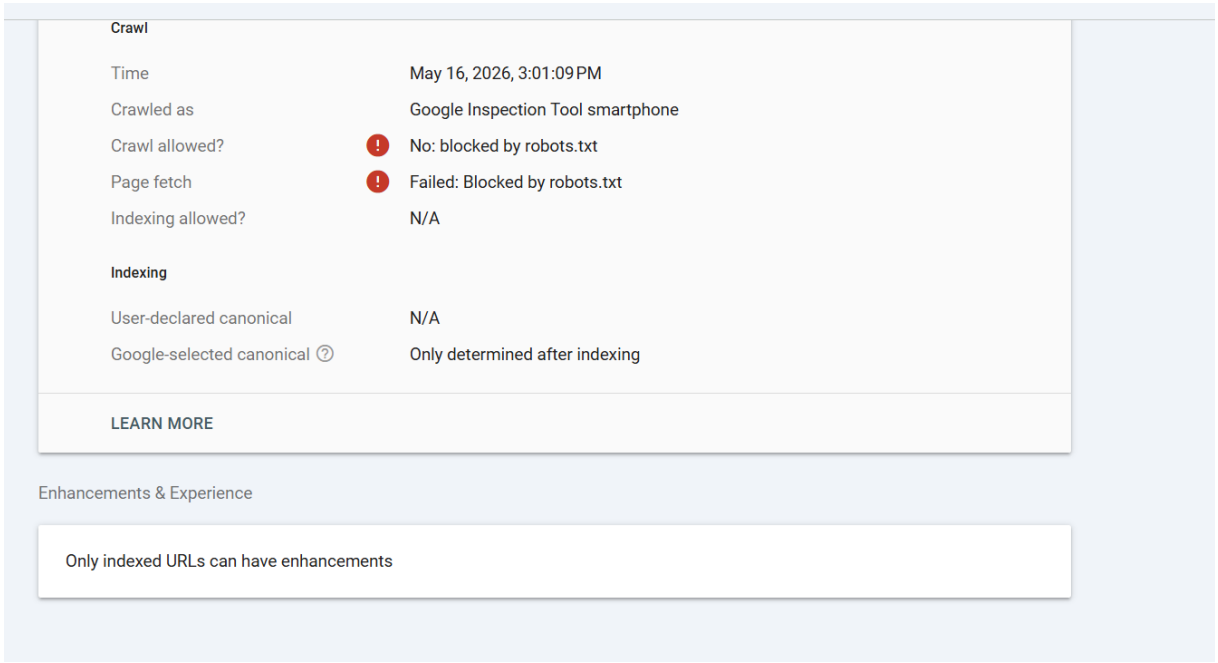

Figure B.4: Blocked by robots.txt (The SEO problem).

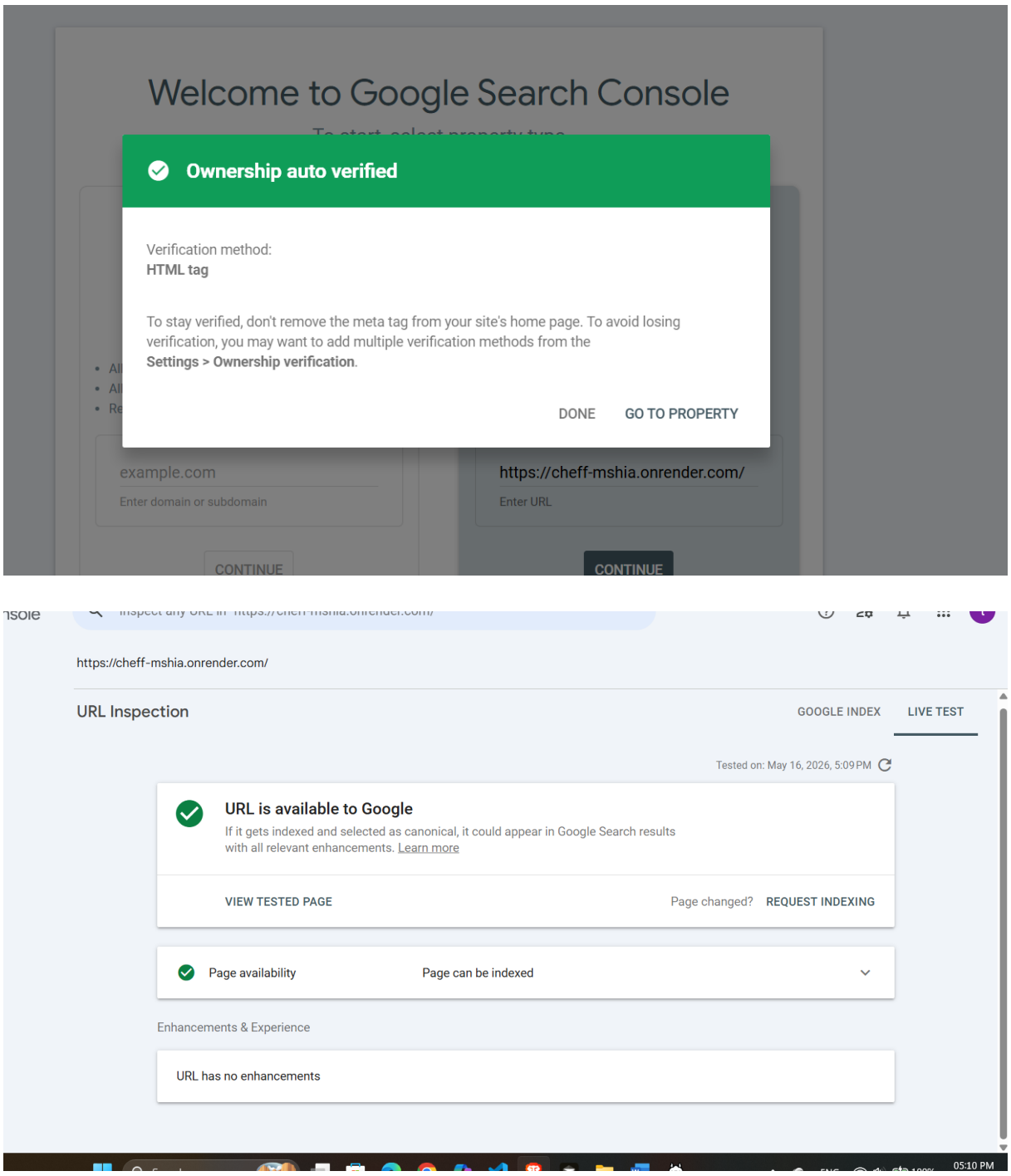

Figure B.5: URL available to Google (The SEO solution).